One-Hot Encoding: Handling Nominal Categorical Data

Contents

1. Overview . . . 2
2. How one-hot encoding works . . . 3
   - 2.1 Why nominal data needs its own technique . . . 3
   - 2.2 One column per category . . . 3
   - 2.3 More categories, more columns . . . 5
3. The dummy variable trap . . . 5
   - 3.1 Dummy variables and the $n - 1$ rule . . . 5
   - 3.2 Multicollinearity: inputs must not depend on each other . . . 6
   - 3.3 Dropping one column loses nothing . . . 7
4. Columns with many categories . . . 8
5. The car data . . . 10
6. One-hot encoding with pandas . . . 11
   - 6.1 get_dummies . . . 11
   - 6.2 Dropping the first column . . . 12
7. One-hot encoding with scikit-learn . . . 13
   - 7.1 Why get_dummies is not used in ML projects . . . 13
   - 7.2 Split first . . . 14
   - 7.3 Creating the encoder . . . 14
   - 7.4 Fit on the training set, transform both . . . 15
   - 7.5 Joining the columns back . . . 16
8. Top categories on the brand column . . . 16
9. Summary . . . 18
10. Sources . . . 18
11. Key terms . . . 19

Where this fits: Pipeline map steps 5 (Engineer features), 8 (Model). See the Course map.



- **Builds on:** Encoding categorical data.
- **Leads to:** Moore-Penrose pseudo-inverse; Simple linear regression; Assumptions of linear regression; Categorical and sparse categorical cross-entropy; Elastic Net; Softmax regression; ANN for classification.
- **Compare with:** Bag of words; Ordinal and label encoding; Linear combinations, span and basis; Word embeddings.

1. Overview

Key point: One-hot encoding (G-1379) turns a nominal column into one 0/1 column per category; we then drop one of those columns, and group rare categories first if there are too many.

Ordinal encoding (G-1404) handled ordinal features, whose categories have an order. A **feature** (G-772) is an input variable (one column of the data table); the **target** (G-1949) is the output we predict; an **observation** (G-1374) is one record (one row). This Note handles the other kind of categorical feature: **nominal** features, whose categories have no order. The technique for them is one-hot encoding.

Figure 1 shows the whole topic. The steps are:

1. one-hot encode the column (section 2);
2. drop one of the new columns to avoid the **dummy variable trap** (G-646) (section 3);
3. when a column has very many categories, keep only the frequent ones (sections 4 and 8).

pandas (section 6) and scikit-learn (section 7) can both do the work.

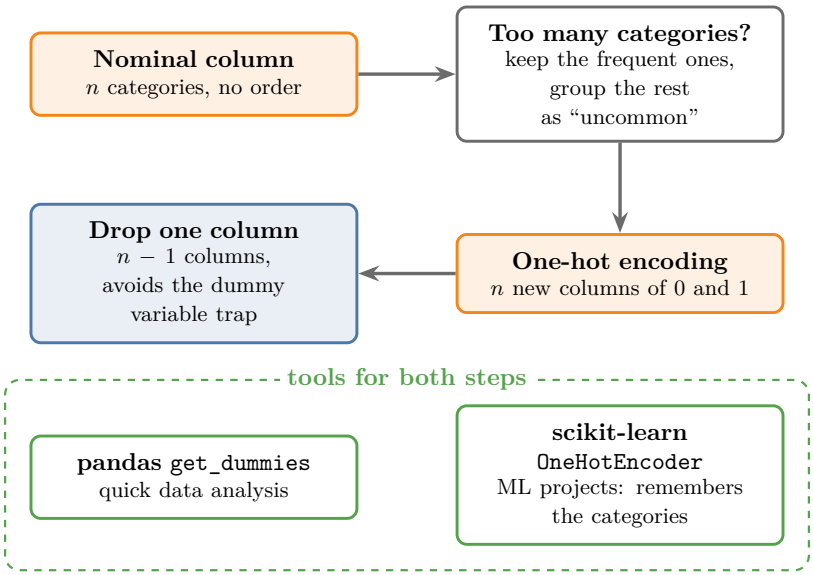


Figure 1: One-hot encoding of a nominal column, step by step, and the two tools that do it

2. How one-hot encoding works

> **Key point:** Each category gets its own column; a row has 1 in the column of its category and 0 everywhere else.

2.1 Why nominal data needs its own technique

> **Key point:** Numbering nominal categories 0, 1, 2 invents an order that does not exist.

ML algorithms need numbers, but real-world categorical data is almost always stored as text. Converting it is our job. For ordinal data we use ordinal encoding (see **ordinal encoding** (G-1404)).

Take a **feature** (G-772) `color` with three categories: Yellow, Blue and Red. Colour is nominal: no colour is greater than another.

If we used ordinal encoding and wrote Yellow = 0, Blue = 1, Red = 2, the algorithm would read Red as "more" than Blue, and Blue as "more" than Yellow. The invented order is false, so we need a different technique.

An everyday picture: numbering colours 0, 1, 2 is like ranking players by the numbers on their shirts. The numbers are only labels, yet a model would treat them as amounts.

2.2 One column per category

> **Key point:** A column with 3 categories becomes 3 columns of 0s and 1s.

One-hot encoding (G-1379) creates one new column for every category of a nominal column. For `color` we get three columns, named after the old column and the category: `color_Y` for Yellow, `color_B` for Blue and `color_R` for Red.

Then we fill them **observation** (G-1374) by observation:

- A Yellow row gets 1 in `color_Y` and 0 in `color_B` and `color_R`.
- A Blue row gets 1 in `color_B` and 0 in the other two.
- A Red row gets 1 in `color_R` and 0 in the other two.

Figure 2 shows the result. In every row exactly one column is "hot" (1), which is where the name comes from.

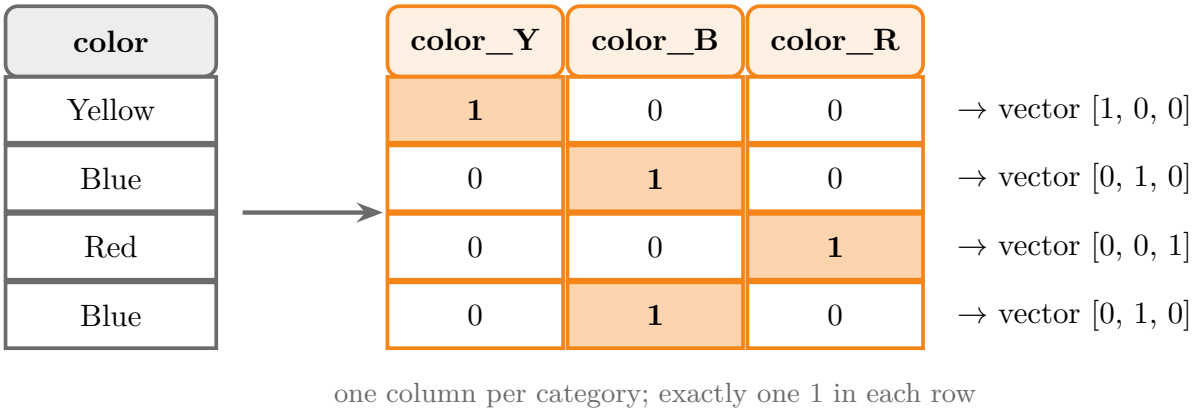


Figure 2: One-hot encoding of the color column: one column per category, one 1 per row

Figure 3 builds the same table step by step. Watch three things in turn: the three new columns appear, one per category; each row then gets its single 1; at the end the first column is dropped, which section 3 explains.

A nominal column: three colours, no order

color
Yellow
Blue
Red
Blue

Row 2 is Blue: 1 in color_B, 0 elsewhere

color	color_Y	color_B	color_R
Yellow	1	0	0
Blue	0	1	0
Red			
Blue			

Row 4 is Blue: 1 in color_B, 0 elsewhere

color	color_Y	color_B	color_R
Yellow	1	0	0
Blue	0	1	0
Red	0	0	1
Blue	0	1	0

Drop the first column: 0, 0 now means Yellow

color	color_B	color_R
Yellow	0	0
Blue	1	0
Red	0	1
Blue	1	0

Figure 3: The color column one-hot encoded step by step: a new column for each category, then one 1 per row, then the first column dropped so that 0, 0 means Yellow. Table build-up after StatQuest, "One-Hot, Label, Target and K-Fold Target Encoding".

In effect, each text value has become a **vector** (G-2081; see what a tensor is), a short list of numbers: Yellow is [1, 0, 0], Blue is [0, 1, 0] and Red is [0, 0, 1]. No vector is bigger than another, so no false order is created. Whenever we meet nominal data in an ML problem, this is what we do. The same trick turned words into vectors in the 3D: text; here it is applied to the categories of a column.

2.3 More categories, more columns

> **Key point:** n categories give n new columns, so a column with 50 categories adds 50 columns.

A natural question: if a column has 50 different categories, do we really make 50 columns? Yes. The vector for each row gets one entry per category.

So the number of columns, the **dimensionality** (G-612) of the data, grows with every category. Section 4 shows how we keep this under control.

3. The dummy variable trap

> **Key point:** After **one-hot encoding** (G-1379) we drop one of the new columns, keeping $n - 1$; the dropped category is still shown by all zeros.

3.1 Dummy variables and the $n - 1$ rule

> **Key point:** The 0/1 columns are called dummy variables; we keep $n - 1$ of them.

The 0/1 columns made by **one-hot encoding** (G-1379) are called **dummy variables** (G-647). In practice we do not keep all of them. With n categories we get n dummy columns, and then we remove one, so that $n - 1$ columns remain.

Usually the first column is the one removed. Removing the second or the last one works just as well; what matters is that exactly one is removed. The reason is a problem called **multicollinearity** (G-1273).

3.2 Multicollinearity: inputs must not depend on each other

> **Key point:** The dummy columns always add up to 1, so any one of them can be calculated from the others; such dependence is called multicollinearity.

The **features** (G-772) are also called **independent variables** (G-937), and the **target** (G-1949) is called the **dependent variable** (G-591): the target depends on the features. As the name says, the features should be independent of each other. There should be no mathematical relationship between them.

Multicollinearity (G-1273) means that some features do have such a relationship: one feature can be calculated from the others. The three colour columns have exactly this problem.

In words: every row has exactly one 1 among the three dummy columns, so the three always add up to 1. Then any one column is 1 minus the other two.

$$Y + B + R = 1 \quad \Longrightarrow \quad Y = 1 - B - R$$

With numbers: for a Blue row, $B = 1$ and $R = 0$:

$$Y = 1 - 1 - 0 = 0$$

For a Red row, $B = 0$ and $R = 1$:

$$Y = 1 - 0 - 1 = 0$$

For a Yellow row, $B = 0$ and $R = 0$:

$$Y = 1 - 0 - 0 = 1$$

The column `color_Y` tells the model nothing that `color_B` and `color_R` do not already say.

A **linear model** predicts a number as a starting value, called the **intercept** (G-960), plus each column multiplied by a **weight** (G-2106), a number the model learns: for example price = intercept + weight × age + weight × mileage. The dependence causes trouble mainly for linear models with an intercept, such as linear regression (a model that predicts a number from a weighted sum of the features) and logistic regression (the same weighted sum, turned

into a probability of a class), which come in later Notes: the dummy columns add up to the intercept column (a column of 1s that the model uses to fit the intercept), so the model cannot be solved uniquely (Kuhn and Johnson 2019, §5.1). Because the dummy columns create it, the problem is called the **dummy variable trap** (G-646).

3.3 Dropping one column loses nothing

> **Key point:** With `color_Y` dropped, the row 0, 0 still means Yellow.

If we remove `color_Y`, can we still tell all three colours apart? Yes (Figure 4):

- Blue is $B = 1, R = 0$.
- Red is $B = 0, R = 1$.
- Yellow is the only colour left when both are 0: $B = 0, R = 0$.

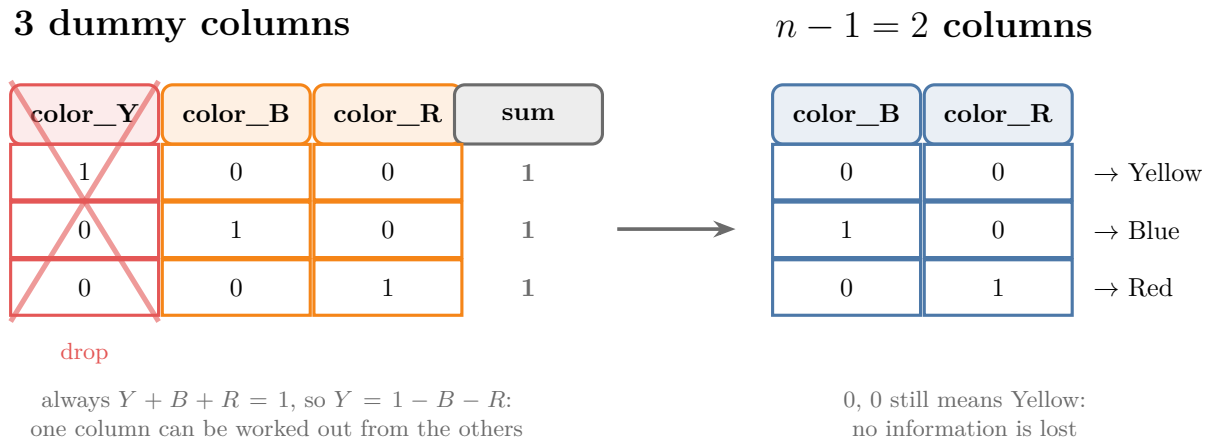


Figure 4: The three dummy columns always add up to 1, so one can be dropped; 0, 0 then stands for Yellow

So $n - 1$ columns are enough to represent n categories. Keeping the n-th one adds no information, only the **multicollinearity** (G-1273). The dropped category is called the **reference category** (G-1650) (or baseline): it is the one shown by all zeros.

> **Extra:** Why multicollinearity confuses a linear model, with small numbers. Suppose yellow cars sell for 5 lakh rupees, blue for 7 and red for 9. A linear model predicts price as $b + w_Y Y + w_B B + w_R R$, where b is a starting value and the w's are **weights** (G-2106) it learns.
>
> - Answer 1: $b = 0$, $w_Y = 5$, $w_B = 7$, $w_R = 9$. A blue car gets:
>
> $$0 + 7 = 7$$
>
> Correct.
>
> - Answer 2: $b = 5$, $w_Y = 0$, $w_B = 2$, $w_R = 4$. A blue car gets:
>
> $$5 + 2 = 7$$

Also correct.

- Answer 3: $b = 100$, $w_Y = -95$, $w_B = -93$, $w_R = -91$. A blue car gets:

$$100 - 93 = 7$$

Still correct.

Because $Y + B + R = 1$, adding any amount to b and taking it off every weight changes nothing, so there are endless "best" answers. The model cannot settle on one, and its weights become unstable and meaningless. With `color_Y` dropped, only one answer is left: $b = 5$ (the yellow price), $w_B = 2$ and $w_R = 4$ (how much more blue and red cars cost than yellow ones).

Extra: Not every model is hurt. Models that are insensitive to linear dependencies between columns, such as regularised linear models, can use all n columns, and tree-based models can work on the categories directly (Kuhn and Johnson 2019, §5.1 and §5.8). Dropping one column is mainly for linear and logistic regression, but it is a safe default.

4. Columns with many categories

Key point: Keep the frequent categories as they are and merge all the rare ones into one category, such as "uncommon", before **one-hot encoding** (G-1379).

Suppose we want to predict the selling price of a second-hand car, and one **feature** (G-772) is `brand`. Brand is nominal: no brand is "greater" than another. But the data has 32 different brands.

One-hot encoding would create 32 columns, one for Maruti Suzuki, one for Hyundai, and so on. The **dimensionality** (G-612) grows, and processing becomes slower.

Figure 5 shows why there is a better way. A few brands, such as Maruti and Hyundai, have thousands of cars, while others, such as Land Rover or Peugeot, have only a handful.

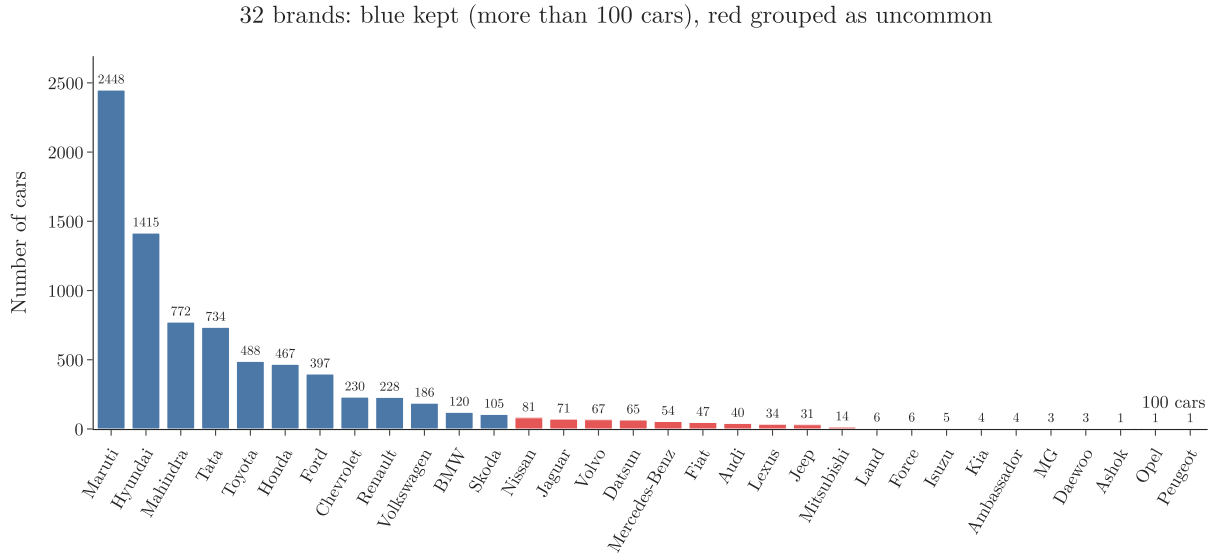


Figure 5: Number of cars per brand in the car data; brands with 100 cars or fewer are grouped as uncommon

So we keep only the most frequent categories and merge all the rare ones into a single new category, called "other" or "uncommon". Here, keeping every brand with more than 100 cars leaves 12 brands plus "uncommon": 13 columns instead of 32. Grouping rare categories is useful whenever some categories are very common and others very rare.

Extra: Target encoding, another answer for columns with very many categories. **Target encoding** (G-2185) replaces each category by the mean of the **target** (G-1949) over the training rows of that category, so the column stays one column. A small case: three Maruti cars sold for 3, 4 and 5 lakh rupees, so Maruti becomes the mean of 3, 4 and 5:

$$(3 + 4 + 5)/3 = 4$$

In the training set of section 7.2, Maruti becomes about 4.02 lakh rupees (the mean selling price of its 1962 cars), Hyundai 4.61 lakh and BMW 41.69 lakh. Two cautions come with it:

- **Rare categories.** Opel has one training car, sold for 68,000 rupees; a mean of one row is not reliable. So the encoder mixes each category's mean with the overall mean (6.41 lakh here), and leans on the overall mean when the category has few rows.
- **Leakage.** The codes are built from the target, so a row's own target must not be used to encode that row; otherwise the model sees the answer during training, a form of **data leakage** (G-535). The standard fix is to split the training set into folds and encode each fold with means from the other folds (k-fold target encoding).

scikit-learn's `TargetEncoder` does both: it shrinks towards the overall mean, and `fit_transform` encodes fold by fold (scikit-learn API, `TargetEncoder`; StatQuest, "One-Hot, Label, Target and K-Fold Target Encoding").

5. The car data

> **Key point:** 8128 used cars with brand, kilometres driven, fuel type, owner and selling price; we predict the selling price.

The dataset has 8128 second-hand cars (**observations** (G-1374)) and five columns:

brand	km_driven	fuel	owner	selling_price
Maruti	145500	Diesel	First Owner	450000
Skoda	120000	Diesel	Second Owner	370000
Honda	140000	Petrol	Third Owner	158000

`selling_price` is the **target** (G-1949), so the task is a regression problem. `km_driven` is already a number. The other three **features** (G-772) are categorical:

- **brand:** 32 categories (Figure 5).
- **fuel:** 4 categories: Diesel (4402 cars), Petrol (3631), CNG (57) and LPG (38).
- **owner:** 5 categories: First Owner, Second Owner, Third Owner, Fourth & Above Owner, and Test Drive Car.

Python: Loading the data and counting the categories of a column.

```
import numpy as np
import pandas as pd

df = pd.read_csv("data/cars.csv")
# how many cars in each fuel category
df["fuel"].value_counts()
# how many different brands: 32
df["brand"].nunique()
```

Figure 6 counts the cars in each category of `fuel` and `owner`. Note the rare categories: only 38 LPG cars and 5 test-drive cars, yet each still gets a whole column.

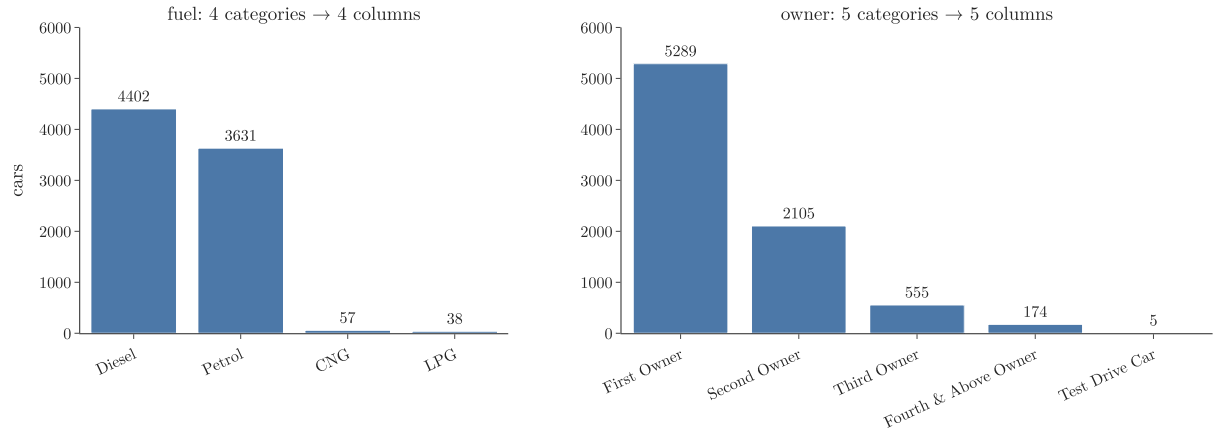


Figure 6: The categories of fuel and owner in the 8128 cars, with the number of cars in each. Every category becomes one one-hot column.

We first one-hot encode `fuel` and `owner`. `brand` gets the many-categories treatment at the end, in Section 8.

> **Extra:** `owner` could also be seen as ordinal: a first-owner car is "newer" than a second-owner one. The category Test Drive Car does not fit neatly into that order, so treating the column as nominal is a reasonable choice. Deciding between nominal and ordinal is often a judgement call like this.

6. One-hot encoding with pandas

> **Key point:** `pd.get_dummies` one-hot encodes the listed columns in one line; `drop_first=True` keeps $n - 1$ columns.

6.1 get_dummies

> **Key point:** Encoding `fuel` and `owner` turns 5 columns into 12.

pandas has a function for **one-hot encoding** (G-1379) called **`get_dummies`** (G-845). We give it the DataFrame and the list of columns to encode.

Figure 7 runs it on the first four cars. Before running it, we can predict the shape. `fuel` is removed and 4 new columns arrive; `owner` is removed and 5 new columns arrive. So 5 columns become 12:

$$5 - 2 + 4 + 5 = 12$$

> **Python:** One-hot encoding two columns with pandas.

```
full = pd.get_dummies(df, columns=["fuel", "owner"],
                      dtype=int)
full.shape
```

```
# (8128, 12)
```

`dtype=int` asks for 0 and 1. Without it, pandas 3 fills the new columns with `True` and `False`.

The untouched columns `brand`, `km_driven` and `selling_price` come first. Then come the new columns, each named **column name, underscore, category name**:

fuel_CNG	fuel_Diesel	fuel_LPG	fuel_Petrol	owner_First Owner	...
0	1	0	0	1	...
0	1	0	0	0	...
0	0	0	1	0	...

6.2 Dropping the first column

Key point: `drop_first=True` removes the first category of each column, giving 10 columns instead of 12.

The result above still contains the **dummy variable trap** (G-646): all n columns are kept for each **feature** (G-772). To keep $n - 1$, we pass the parameter `drop_first=True`.

Python: $n - 1$ columns per feature.

```
k_minus_1 = pd.get_dummies(df, columns=["fuel", "owner"],
                           drop_first=True, dtype=int)
k_minus_1.shape
# (8128, 10)
```

One column fewer per feature: `fuel_CNG` and `owner_First Owner` are gone. The categories are sorted alphabetically, so these are the first ones. A CNG car is now the row with 0 in all three remaining fuel columns.

The car data: fuel and owner are text

brand	km_driven	fuel	owner
Maruti	145500	Diesel	First Owner
Skoda	120000	Diesel	Second Owner
Honda	140000	Petrol	Third Owner
Hyundai	127000	Diesel	First Owner

owner → 5 columns: 12 columns in all

brand	km_driven	fuel_CNG	fuel_Diesel	fuel_LPG	fuel_Petrol	own_1st	own_4th+	own_2nd	own_test	own_3rd
Maruti	145500	0	1	0	0	1	0	0	0	0
Skoda	120000	0	1	0	0	0	0	1	0	0
Honda	140000	0	0	0	1	0	0	0	0	1
Hyundai	127000	0	1	0	0	1	0	0	0	0

drop_first=True: fuel_CNG and owner_First Owner removed, 10 columns

brand	km_driven	fuel_Diesel	fuel_LPG	fuel_Petrol	own_4th+	own_2nd	own_test	own_3rd
Maruti	145500	1	0	0	0	0	0	0
Skoda	120000	1	0	0	0	1	0	0
Honda	140000	0	0	1	0	0	0	1
Hyundai	127000	1	0	0	0	0	0	0

Figure 7: get_dummies on the first four cars: the text columns fuel and owner become 4 and 5 columns of 0s and 1s (12 columns in all); drop_first=True then removes fuel_CNG and owner_First Owner (10 columns). Orange cells are the 1s. Owner columns are shortened, for example own_2nd for owner_Second Owner.

7. One-hot encoding with scikit-learn

> **Key point:** In an ML project we use scikit-learn's `OneHotEncoder`, because it remembers the categories it learned and always produces the same columns.

7.1 Why get_dummies is not used in ML projects

> **Key point:** `get_dummies` only sees the rows it is given, so new data can produce different columns.

`get_dummies` is handy for data analysis, but it is not used inside an ML project. The function does not remember which categories it saw, or which column it put at which position. Run it on different data and the result can change.

Figure 8 shows the problem. The model was trained on 4 fuel columns.

New data that happens to hold only Diesel and Petrol cars gives just 2 columns with `get_dummies`, so the shapes no longer match. `OneHotEncoder` learned all 4 categories during `fit` and always returns the same 4 columns.

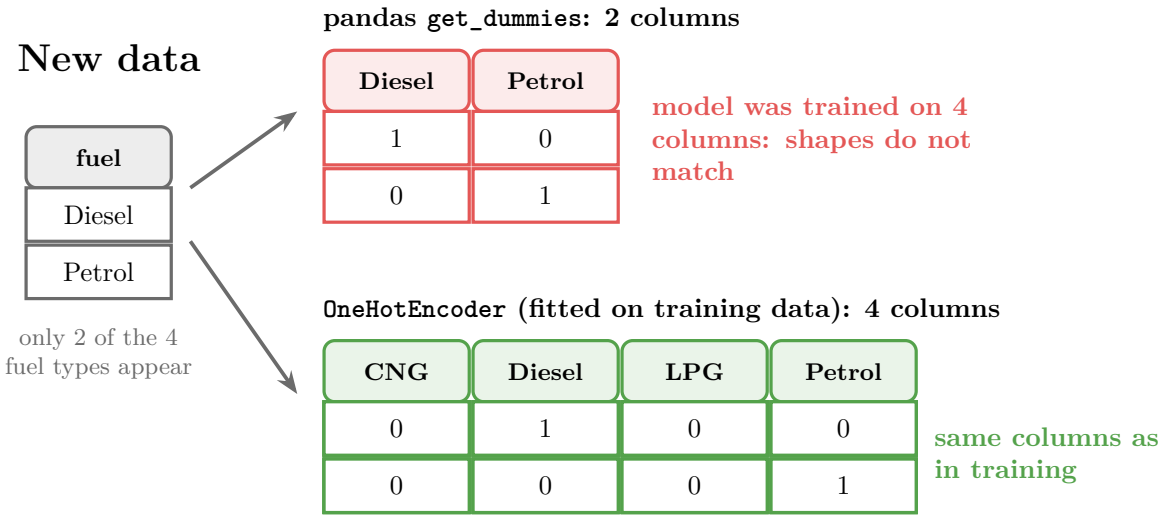


Figure 8: On new data with only two fuel types, get_dummies makes 2 columns; a fitted OneHotEncoder still makes the 4 columns the model expects

7.2 Split first

> **Key point:** As always, the train-test split comes before the encoding.

The encoder must learn its categories from the training rows only, so the test set stays unseen, as in split before scaling. The first four columns are the **features** (G-772) and `selling_price` is the target. We hold back 20% of the **observations** (G-1374) for testing: 6502 training rows and 1626 test rows.

Python: Splitting the car data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df.iloc[:, 0:4],    # brand, km_driven, fuel, owner
    df.iloc[:, -1],     # selling_price
    test_size=0.2, random_state=2)
```

7.3 Creating the encoder

> **Key point:** `drop="first"` gives $n-1$ columns, `sparse_output=False` gives a normal array, and `dtype` sets the number type.

scikit-learn's class for this is **`OneHotEncoder`** (G-1390). Three parameters are useful here:

- **`drop="first"`:** drop the first category of each column, as `drop_first=True` did in pandas. The value must be the text `"first"`; `drop=True` gives an error.
- **`sparse_output=False`:** return a normal NumPy array (explained below).

- **`dtype=np.int32`:** fill the array with whole numbers, 0 and 1, instead of the default decimals 0.0 and 1.0.

Python: Creating the encoder.

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(drop="first", sparse_output=False,
                    dtype=np.int32)
```

By default, `OneHotEncoder` returns a **sparse matrix** (G-1843). Most entries of a one-hot result are 0, so a sparse matrix saves memory by storing only the positions and values of the non-zero entries. To see it as a normal table we would call `.toarray()` on it; `sparse_output=False` gives the normal array straight away.

Extra: In older scikit-learn versions this parameter was called `sparse`. The parameter was renamed `sparse_output` in version 1.2, and the old name was removed in 1.4 (scikit-learn release notes, 1.2 and 1.4). Likewise, `get_feature_names` is now `get_feature_names_out`.

7.4 Fit on the training set, transform both

Key point: We encode only `fuel` and `owner`, so we pass just those two columns; the result has 7 columns.

Unlike `get_dummies`, we do not apply the encoder to the whole table. `brand` stays as it is for now, and `km_driven` is already a number. So we pass only the two columns to encode, with double brackets `X_train[["fuel", "owner"]]`.

Python: Encoding `fuel` and `owner`.

```
# learn the categories from the training set, then encode it
X_train_new = ohe.fit_transform(X_train[["fuel", "owner"]])
# encode the test set with the same categories
X_test_new = ohe.transform(X_test[["fuel", "owner"]])
X_train_new.shape
# (6502, 7)
```

`fit_transform` does `fit` and then `transform` in one call.

Seven columns:

$$(4-1)+(5-1)=3+4=7$$

The encoder can tell us their names and the categories it learned:

Python: Looking inside the fitted encoder.

```
ohe.categories_
# [array(['CNG', 'Diesel', 'LPG', 'Petrol'],
#  array(['First Owner', 'Fourth & Above Owner',
```

```
#          'Second Owner', 'Test Drive Car',
#          'Third Owner'])]
ohe.get_feature_names_out()
# ['fuel_Diesel', 'fuel_LPG', 'fuel_Petrol',
#  'owner_Fourth & Above Owner', 'owner_Second Owner',
#  'owner_Test Drive Car', 'owner_Third Owner']
```

The first training row is a Diesel car with a first owner. The row becomes [1, 0, 0, 0, 0, 0, 0]: 1 in `fuel_Diesel`, and all zeros for owner, because First Owner is the dropped category.

> **Extra:** If the test set holds a category never seen in training (say, an Electric car), `transform` stops with an error. `handle_unknown="ignore"` writes all zeros for it instead. Combined with `drop="first"`, those zeros look exactly like the dropped category (CNG), so the unknown car is treated as CNG; scikit-learn prints a warning, but the row is still all zeros.

7.5 Joining the columns back

> **Key point:** We put `brand` and `km_driven` back next to the 7 new columns with `np.hstack`, giving 9 columns.

The encoded array holds only the fuel and owner information. To get all the **features** (G-772), we join it with the columns we did not encode.

> **Python:** Stacking arrays side by side.
> ```
> X_train_full = np.hstack(
>     (X_train[["brand", "km_driven"]].values, X_train_new))
> X_train_full.shape
> # (6502, 9)
> ```
> `.values` turns the two DataFrame columns into a NumPy array. `np.hstack` ("horizontal stack") joins arrays with the same number of rows, side by side.

Stacking by hand works, but it is clumsy: pull out some columns, encode them, then glue everything back together. The **column transformer** (G-415), covered in the next Note, applies a different transformer to each column in a single step.

8. Top categories on the brand column

> **Key point:** Brands with 100 cars or fewer are replaced by "uncommon"; **one-hot encoding** (G-1379) then gives 13 columns instead of 32.

Back to `brand` and its 32 categories (Figure 5). Figure 9 shows the grouping before and after. We count the cars per brand and pick a threshold of 100. Every brand at or below it is replaced by the word `uncommon`, which affects 20 brands, from Nissan (81 cars) down to Peugeot (1 car).

Python: Grouping rare brands, then one-hot encoding.

```
counts = df["brand"].value_counts()
threshold = 100
# names of the brands with 100 cars or fewer
repl = counts[counts <= threshold].index
brand_top = df["brand"].replace(repl, "uncommon")
pd.get_dummies(brand_top, dtype=int).shape
# (8128, 13)
```

`counts <= threshold` gives True or False for each brand; `counts[...]` keeps only the True ones, and `.index` takes their names.

The 13 columns are the 12 frequent brands (BMW, Chevrolet, Ford, Honda, Hyundai, Mahindra, Maruti, Renault, Skoda, Tata, Toyota, Volkswagen) plus `uncommon`. The `uncommon` column is rarely 1: only 538 of the 8128 cars, about 7%, fall into it.

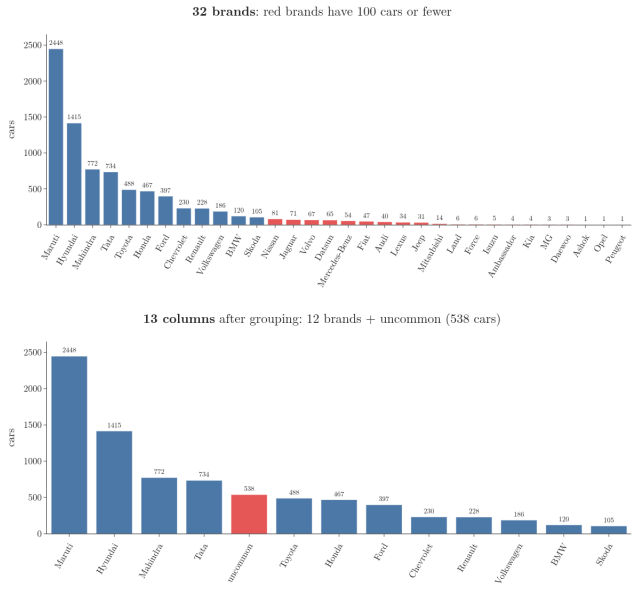


Figure 9: The 32 brands of the car data, then the 13 columns left after the 20 brands with 100 cars or fewer (red) are replaced by uncommon, which holds 538 cars.

Extra: `OneHotEncoder` can do this grouping itself. With `min_frequency=101`, categories seen fewer than 101 times in the training set are merged into one column, `brand_infrequent_sklearn`; `max_categories=13` instead keeps at most 13 columns. Adding `handle_unknown="infrequent_if_exist"` sends brands never seen before into the same column (scikit-learn API docs, `OneHotEncoder`). The encoder's grouping is better than replacing by hand: the counting happens on the training set only, as the split-first rule requires, and the encoder remembers it. On this training set Skoda has only 81 cars, so it gets grouped too.

9. Summary

	pandas `get_dummies`	scikit-learn `OneHotEncoder`
Typical use	quick data analysis	ML projects
Remembers the categories	no	yes, after `fit` (`categories_`)
Keep $n-1$ columns	`drop_first=True`	`drop="first"`
Output	DataFrame (bool columns in pandas 3, or `dtype=int`)	sparse matrix, or array with `sparse_output=False`
Unknown category later	silently different columns	error, or zeros with `handle_unknown="ignore"`
Rare categories	replace by hand, then encode	`min_frequency` or `max_categories`

- Nominal categories have no order, so we do not number them; we give each category its own 0/1 column.
- n categories give n **dummy variables** (G-647), and exactly one of them is 1 in each row, so a column with 50 categories adds 50 columns.
- The dummy columns always add up to 1, so one depends on the others: **multicollinearity** (G-1273), the dummy variable trap. A linear model with an intercept then has endless equally good weights and cannot settle on one.
- Dropping one column fixes it and loses nothing: all zeros stand for the dropped category.
- For a column with many categories, keep the frequent ones and merge the rare ones into "uncommon", so the number of columns stays small (13 instead of 32 for `brand`).
- In ML projects, split first, fit `OneHotEncoder` on the training set, and transform both sets, because the fitted encoder remembers its categories and always returns the columns the model expects, while `get_dummies` can return different columns on new data.
- So a nominal column becomes one 0/1 column per category, minus one to avoid the trap, with rare categories grouped first when there are too many.

10. Sources

Built from

- CampusX, "One Hot Encoding | Handling Categorical Data | Day 27 | 100 Days of Machine Learning", YouTube, https://www.youtube.com/watch?v=U5oCv3JKWKA
- StatQuest with Josh Starmer, "One-Hot, Label, Target and K-Fold Target Encoding, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=589nCGeWG1w (the table build-up of Figure 3; target encoding in section 4)

Other references

- Kuhn, M. and Johnson, K. (2019). *Feature Engineering and Selection: A Practical Approach for Predictive Models*. CRC Press. §5.1 Creating Dummy Variables for Unordered Categories; §5.8 Factors versus Dummy Variables in Tree-Based Models. feat.engineering.

- scikit-learn API reference. `sklearn.preprocessing.OneHotEncoder` and `sklearn.preprocessing.TargetEncoder`. scikit-learn.org.
- scikit-learn release notes. Versions 1.2 and 1.4. scikit-learn.org/stable/whats_new.

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
One-hot encoding (G-1379)	Replacing a nominal column by one 0/1 column per category, with a single 1 in each row; also used to represent words.
Dummy variable trap (G-646)	The problem caused by keeping all n dummy columns of a category: they always add up to 1, so any one can be calculated from the others (multicollinearity) and a linear model's weights become unstable. Dropping one column ($n - 1$ kept) avoids it.
Dimensionality (G-612)	The number of features (columns) in the data; one-hot encoding raises it by one column per category, and too high a dimensionality brings the curse of dimensionality.
Dummy variable (G-647)	One of the 0/1 columns made by one-hot encoding; each marks whether a row belongs to one category, so a model can use a categorical column as numbers.
Multicollinearity (G-1273)	A relationship between input columns, so that one can be calculated (or largely predicted) from the others; it makes a linear model's coefficients unstable and hard to interpret.
Reference category (G-1650)	The category whose dummy column is dropped; it is shown by all zeros.
Target encoding (G-2185)	Replacing each category by the mean of the target for that category, learned on training rows only.
get_dummies (G-845)	pandas function that turns each category into its own 0/1 column (one-hot encoding); `drop_first=True` keeps $n - 1$ of them.
OneHotEncoder (G-1390)	scikit-learn's class for one-hot encoding; remembers the categories it learned.

Term	Meaning
Sparse matrix (G-1843)	A table stored as only its non-zero entries, to save memory.
get_feature_names_out (G-846)	`OneHotEncoder` method that returns the names of the new columns.
handle_unknown (G-875)	`OneHotEncoder` parameter that decides what happens to categories never seen in training.
min_frequency (G-1218)	`OneHotEncoder` parameter that merges rare categories into one column.
sparse_output (G-1847)	The `OneHotEncoder` setting that picks the output format: `False` returns a normal NumPy array instead of a sparse matrix (a table stored as only its non-zero entries).
Top categories (G-1987)	Keeping only the most frequent categories and merging the rest into one "uncommon" category.
Ordinal encoding (G-1404)	Replacing ordered categories by 0, 1, 2, ... in their order; for input columns.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Observation (G-1374)	One record of a dataset: one row of the data table.
Vector (G-2081)	A list of numbers: a 1D tensor.
Independent variables (G-937)	Another name for the features: the input columns X that a model uses to predict the output.
Dependent variable (G-591)	Another name for the target, the output y that a model predicts; it is called dependent because it depends on the features.
Intercept (G-960)	The line's value when the input is 0; b in $y = mx + b$.
Weight (in a network) (G-2106)	A number the network learns on the connection between two neurons; it sets how strongly one neuron's output counts in the next neuron's sum.
Data leakage (G-535)	Information from the test rows reaching the model during training, for example when a scaling or feature-selection step is fitted on all the data before the split; the test scores then come out higher than the model will really do on new data.

Term	Meaning
Column transformer (G-415)	A scikit-learn class that applies different transformations to different columns at once and joins the results.